
DTU

62550 Brugerinteraktion og udvikling på mobile enheder

Mobiludvikling

Oversigt

- ❑ Opsummering
- ❑ Activity cyklus
- ❑ Software Arkitektur
- ❑ Flow
- ❑ Parallel programming principper
- ❑ Coroutiner

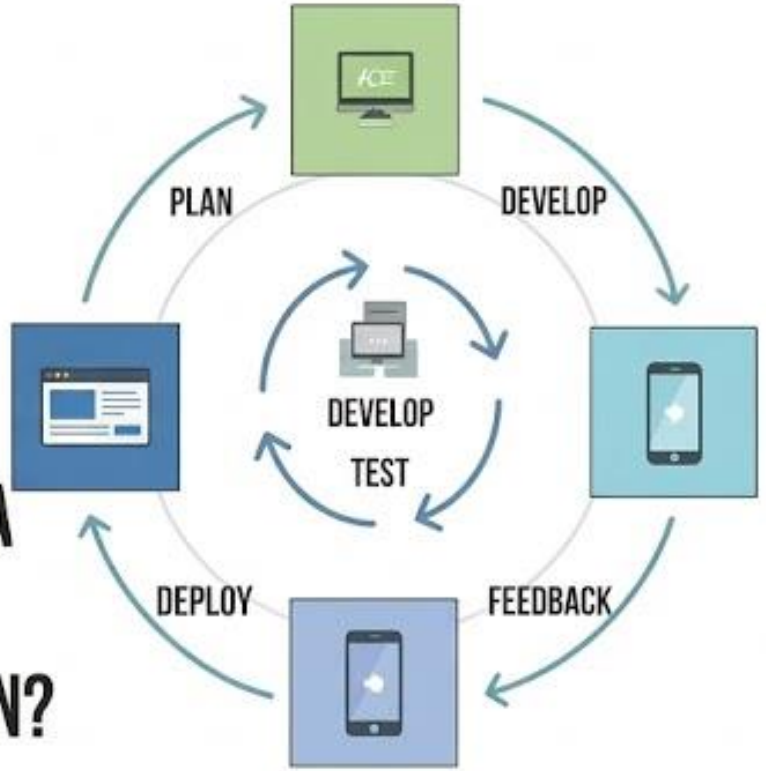
Opsummering

DESIGN THINKING



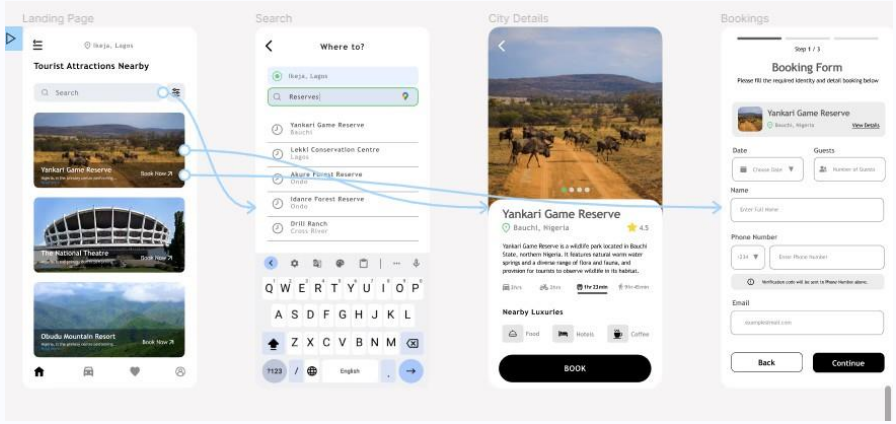
HVØRDAN GÅR VI FRA ABSTRAKT IDÉ TIL KONKRET PRODUKTION?

AGILE

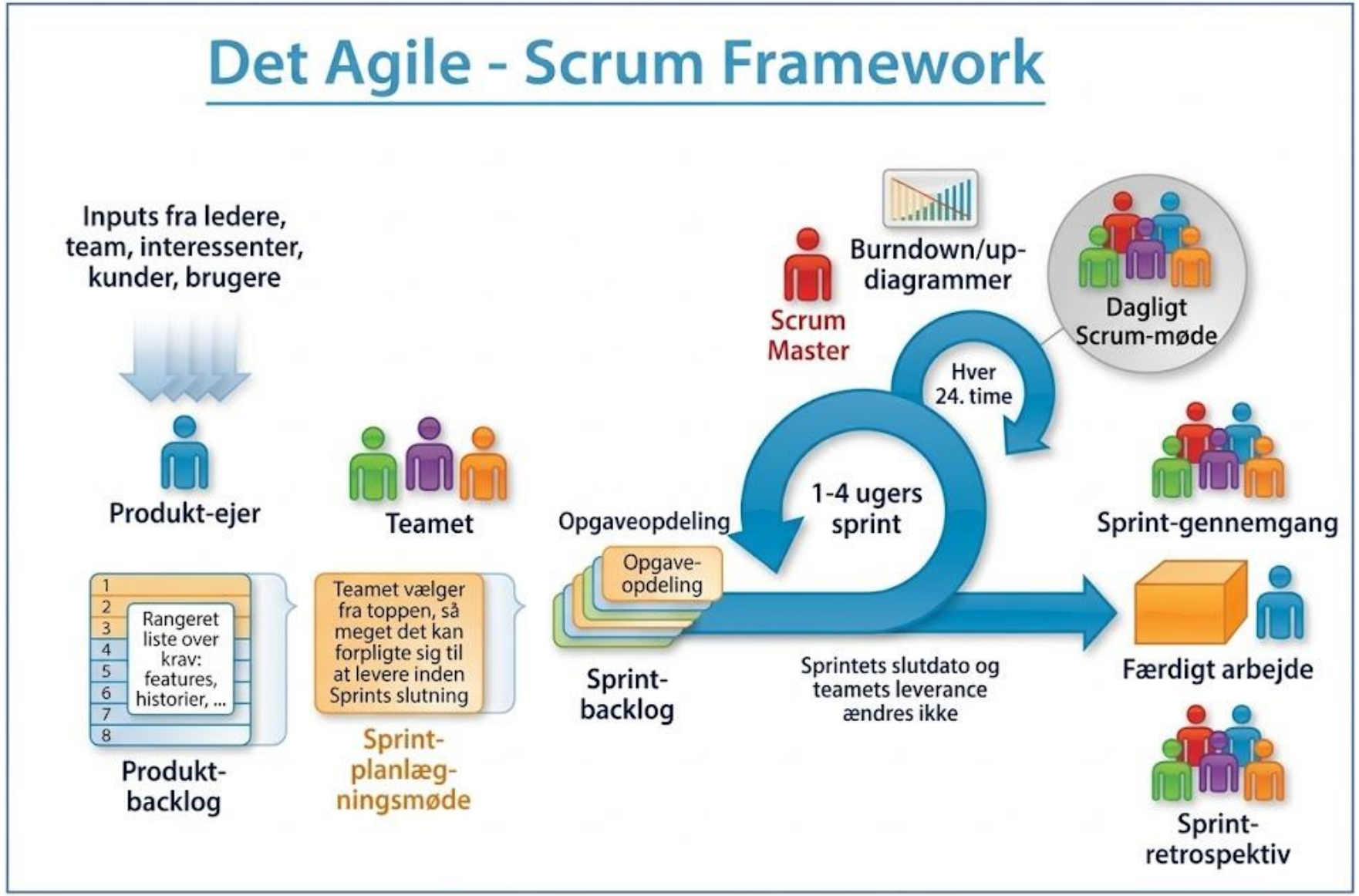


Brugertest udfald

User Story + Acceptance Kriterier +



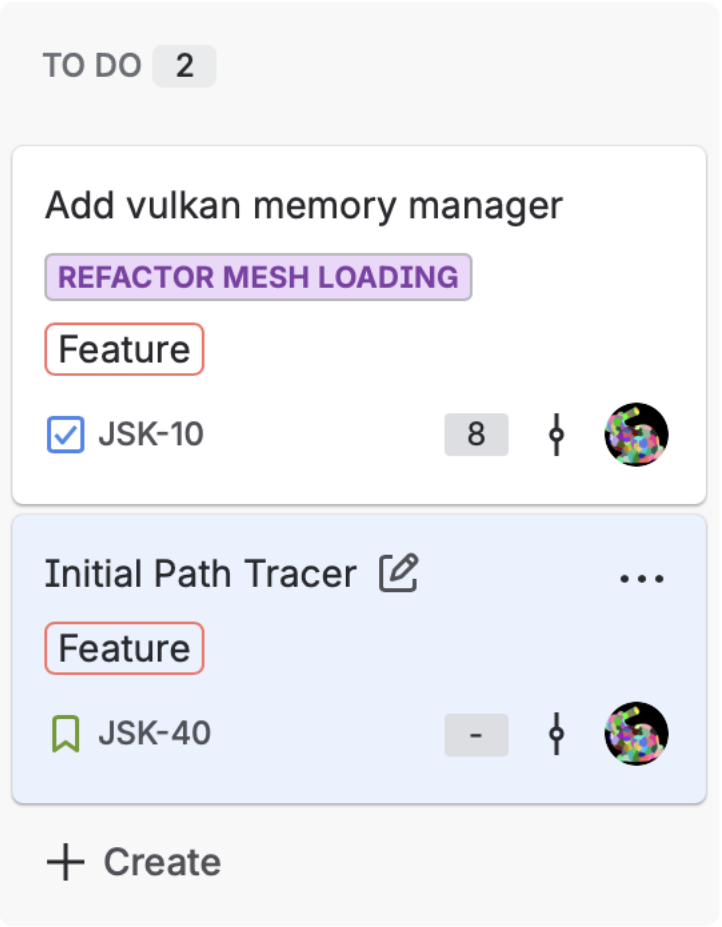
https://www.banani.co/definitions/figma-prototype



https://www.c-sharpcorner.com/UploadFile/d9c992/the-agile-scrum-framework/

Tickets

- En **ticket** ("issue", "item") er den mindste, atomare enhed af arbejde i et agilt system (som Jira, Trello, etc.).
- Det er en enkelt opgave, der skal spores, tildeles og fuldføres.
- Alt arbejde – fra en simpel opgave til en kompleks fejl – starter som et ticket.
- Tickets sikrer **transparens** og **sporbarhed** i teamets workflow.



Fra småt til Stort

De fleste agile teams bruger et hierarki til at organisere arbejde:

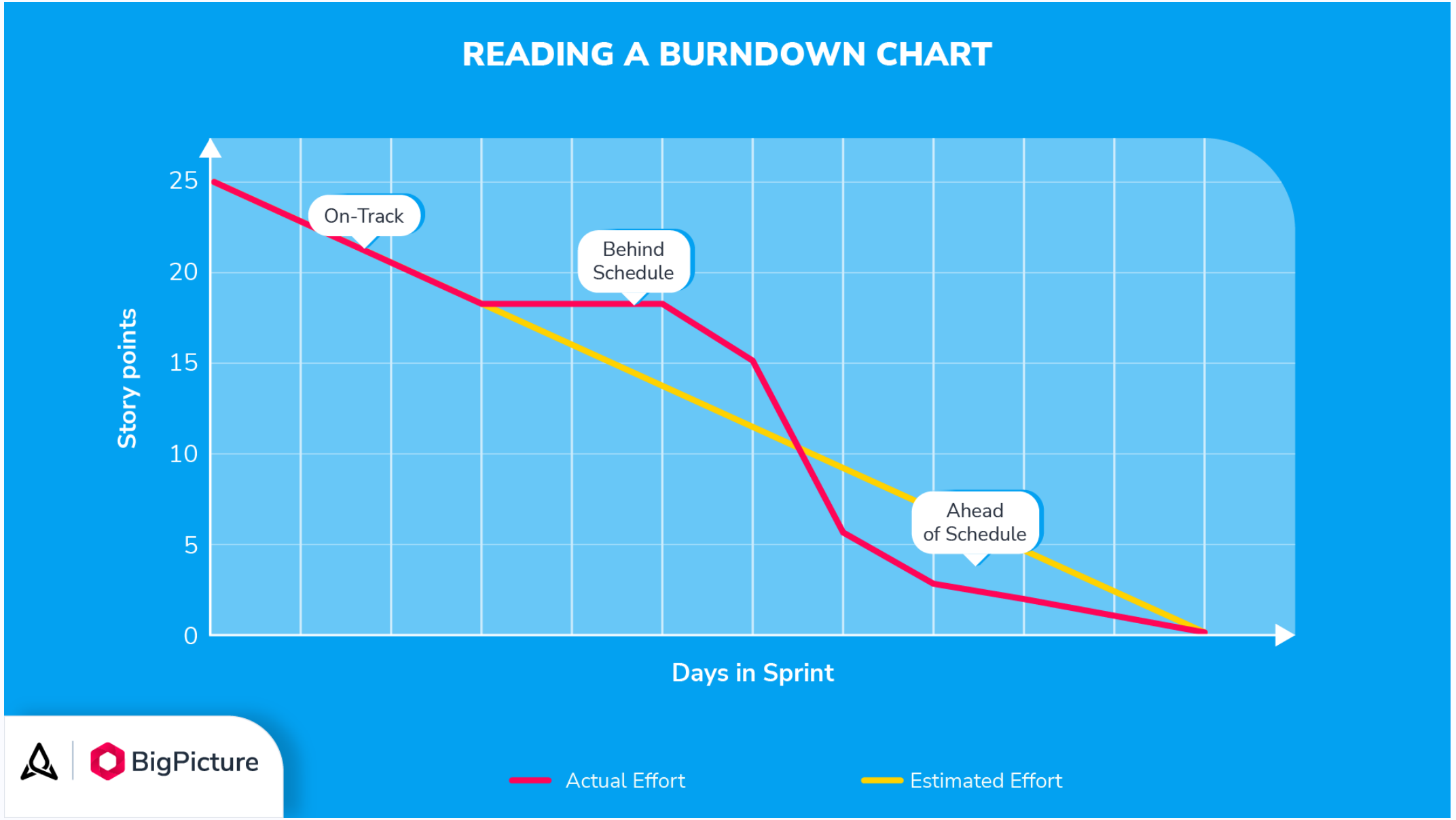
- **Epic** (Det store mål)
  - **Feature** (En brugbar del af Epic'et)
    - **User Story** (Et specifikt brugerbehov)
      - **Ticket/Sub-task** (En teknisk delopgave)

Side-items:

- **Bug** (Fejl),
- **Spike** (Undersøgelse)

Ticket Estimering

- **Formål:** Ikke at fastsætte en deadline, men at skabe **fælles forståelse**.
- Estimering tvinger teamet til at diskutere en opgaves **kompleksitet**.
- **Det hjælper med at:**
  - Afdække skjulte antagelser ("Jeg troede, vi bare...")
  - Identificere usikkerhed og risici.
  - Skabe en prognose for, hvor meget arbejde teamet kan nå (se Sprint Velocity).



- https://bigpicture.one/blog/sprint-burndown-chart-101/

Activity Lifecycle

Hvad er en Activity?

Activity

- En **Activity**(aktivitet)(android/KMP),**View Controller**(UIKit), **View**(SwiftUI) er en af de mest grundlæggende byggeklodser.
- Moderne applikationsudvikling med **Jetpack Compose** følger paradigmet **Single-Activity Architecture**
- **Jetpack Compose destinations** bruges til at navigere rundt mellem forskellige ”destinationer” i applikationen.
- **Destinationerne** er dekoreret med *@Composable*

Activity

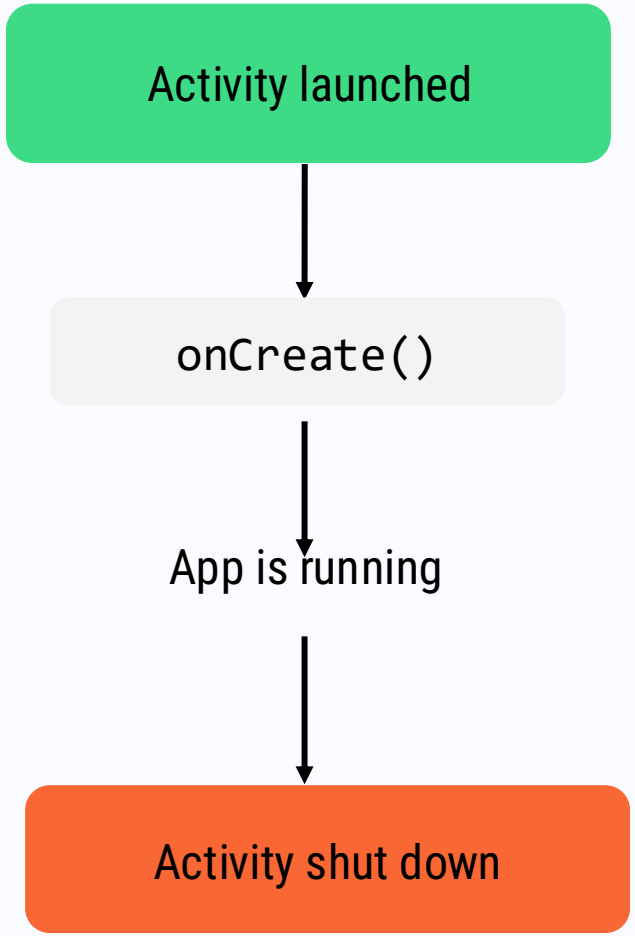
En app består normalt af én aktivitet som står for følgende:

- **Brugergrænseflade (UI):** Er ansvarlig for at tegne en brugergrænseflade (UI) på skærmen.
- **Livscyklus (Lifecycle):** En Activity har en kompleks livscyklus. Den er ikke bare "tændt" eller "slukket". Den gennemgår forskellige tilstande, som appen skal håndtere korrekt for at give en god brugeroplevelse og undgå at miste data.
- **Entry Point:** En aktivitet er defineret som "hovedaktiviteten" (main activity). Det er den skærm, der vises, når brugeren starter din app fra telefonens startskærm.

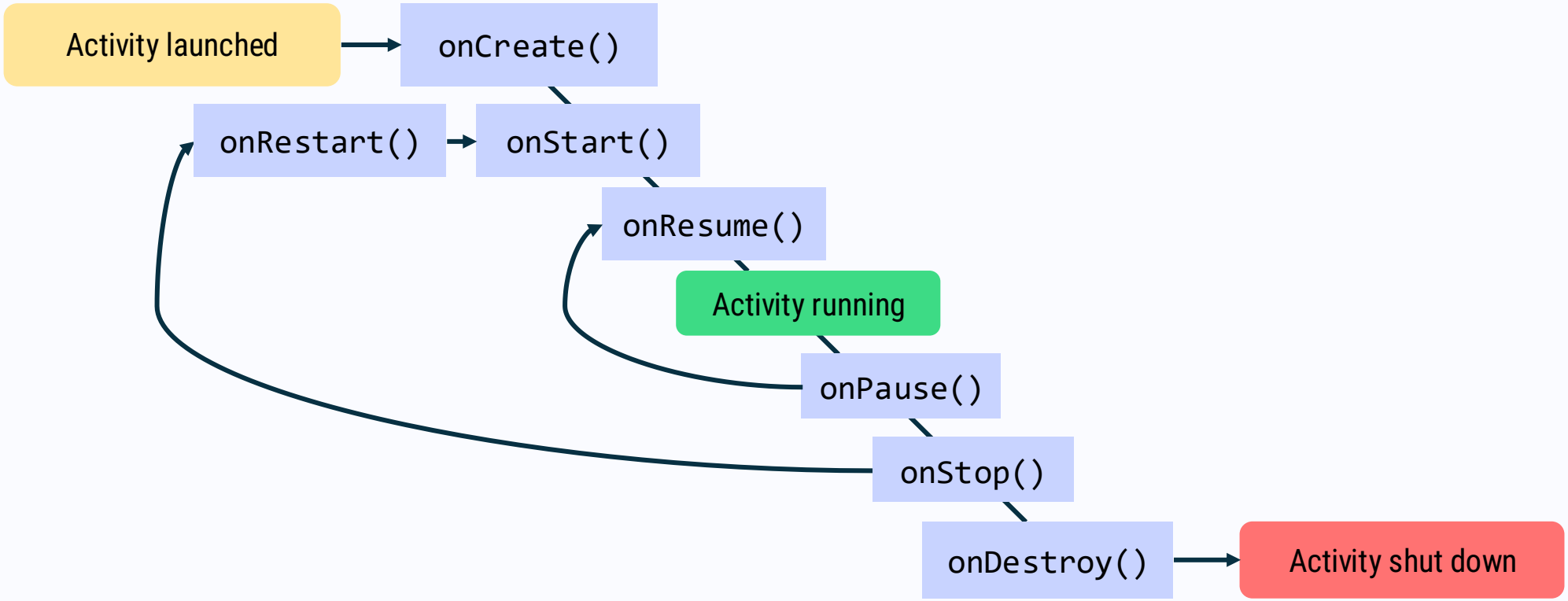
Activity lifecycle

- Bevar brugerdata og tilstand hvis:
  - Brugeren midlertidigt forlader appen og vender tilbage
  - Brugeren afbrydes(for eksempel af et telefonopkald)
  - Brugeren roterer mobiltelefonen
- Undgå memory leaks og app crashes.

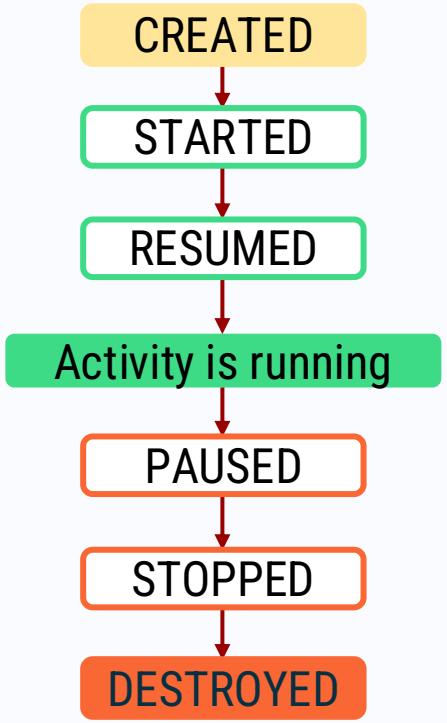
Activity lifecycle



Activity lifecycle



Activity States




```kotlin
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
        HelloLifecycleTheme {
            // A surface container using the 'background'
            Surface(modifier = Modifier.fillMaxSize(),
                color = MaterialTheme.colors.background) {
                Greeting( name: "Android")
            }
        }
    }
    Log.d(TAG, msg: "onCreate")
}

override fun onStart() {
    super.onStart()
    Log.d(TAG, msg: "onStart")
}

override fun onRestart() {
    super.onRestart()
    Log.d(TAG, msg: "onRestart")
}

override fun onResume() {
    super.onResume()
    Log.d(TAG, msg: "onResume")
}

override fun onPause() {
    super.onPause()
    Log.d(TAG, msg: "onPause")
}

override fun onStop() {
    super.onStop()
    Log.d(TAG, msg: "onStop")
}

override fun onDestroy() {
    super.onDestroy()
    Log.d(TAG, msg: "onDestroy")
```

onCreate()

- Activity oprettes, og andet initialiseringsarbejde køres
- Skal implementeres af udvikleren
- Inflater brugergrænseflade (UI) og udfør andet app-opstartslogik

onStart()

- Activity bliver synlig for brugeren
- Kald efter activity:
  - `onCreate()`

    or
  - `onRestart()` Hvis aktiviteten tidligere er blevet stoppet

onResume()

- Aktiviteten får inputfokus:
  - Brugeren kan interagere med aktiviteten
- Aktiviteten forbliver I genoptaget tilstand(resumed state), indtil systemet udløser, at aktiviteten sættes på pause

onPause()

- Aktiviteten har mistet fokus (er ikke længere I forgrunden)
- Aktiviteten er stadig synlig, men brugeren interagerer ikke aktivt med den
- modstykke til `onResume()`

onStop()

- Aktiviteten er ikke længere synlig for brugeren
- Frigiv ressourcer, der ikke længere er nødvendige
- Gem enhver vedvarende tilstand(persistent state), som brugeren er I gang med at redigere, så de ikke mister deres arbejde

onDestroy()

Brugeren forventer, at UI-tilstanden(brugergrænsefladens tilstand) forbliver den samme efter en konfigurationsændring, eller hvis appen lukkes, mens den er i baggrunden

- Aktiviteten ødelægges og genstartes, eller appen lukkes, og aktiviteten startes
- Gem brugerdata, der er nødvendig for at gendanne appen og aktivitetens livscyklus-ændringer:
  - Brug `Bundle` leveret af `onSaveInstanceState()`.
  - `onCreate()` modtager `Bundle'en` som et argument, når aktiviteten skabes igen

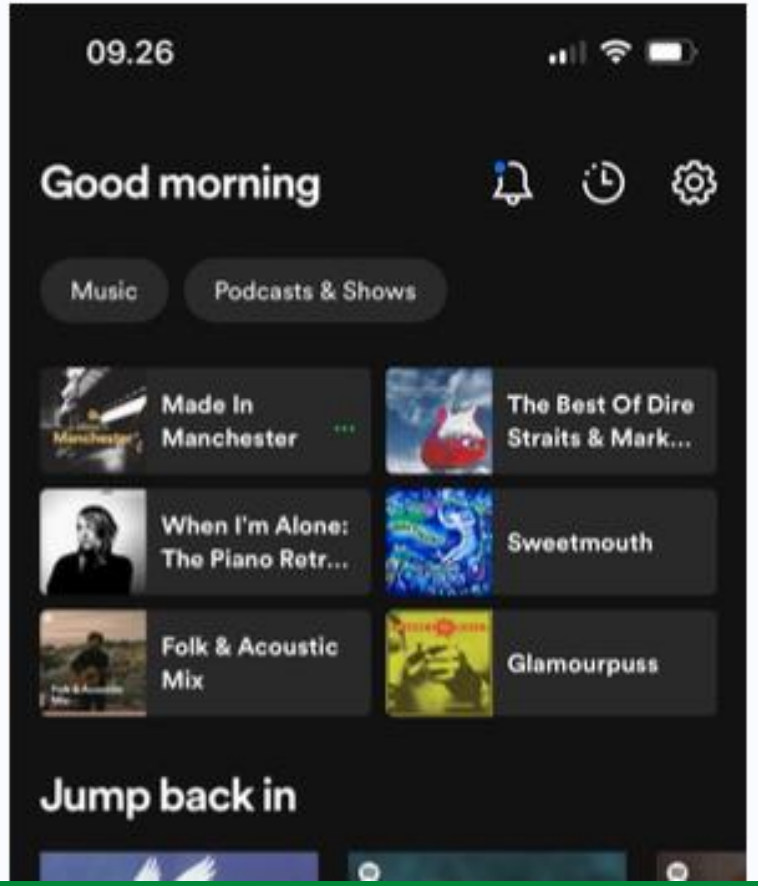
Opsummering af activity states

State	Callbacks	Description
Created	onCreate()	Aktiviteten initialiseres
Started	onStart()	Aktiviteten er synlig for brugeren
Resumed	onResume()	Aktiviteten har input fokus
Paused	onPause()	Aktiviteten har mistet input fokus
Stopped	onStop()	Aktiviteten er ikke synlig længere
Destroyed	onDestroy()	Aktiviteten destrueres

State og State hoisting

State

- State i en applikation, er en hvilken som helst værdi som ændres over tid, som f.eks. tekst, eller indholdet af en liste
- State er den data der vises i brugergrænsefladen



Hvad er en composable?

Composable

- En **Composable** er den centrale byggeklods i **Jetpack Compose**, som er Androids moderne, deklarative UI-framework. (**View** i SwiftUI)
- Tænk på det som en lille, genanvendelig UI-del.
- I praksis er en Composable en **Kotlin-funktion**, der er markeret med annotationen @Composable.
- Annoteringen *beskriver*, hvordan en del af brugergrænsefladen (UI) skal se ud. Den tager data ind og "udsender" (emits) en UI komponent.

Composable

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello $name!")
}

@Composable
fun MyScreen() {
    Column {
        Greeting(name = "Android")
        Button(onClick = { /* ... */ }) {
            Text("Tryk her")
        }
    }
}
```

Activity og Composable

1.Activity: Dette er "skærmen" eller "containeren". Den er en tung komponent, der er ejet af Android-systemet. Den håndterer den overordnede livscyklus (når appen går i baggrunden, roterer osv.) og fungerer som indgangspunktet til brugergrænsefladen(UI).

2.@Composable: *kun* en UI-byggeklods. Den beskriver, hvordan en del af UI'en skal se ud og opføre sig. Den bor *inde i* en Aktivitet og ejer ikke selv skærmen.

Bemærk: Du kan ikke have en Composable på skærmen uden en Aktivitet (eller et Fragment) til at "hoste" den.

State og composition

- Brugergrænsefladen opdateres kun når den samme composable kaldes med nye argumenter
- En composable skal kaldes eksplicit med nye argumenter for at blive opdateret ordentligt
- En recomposition finder sted hver gang applikationens state ændrer sig

State

```
42  @Composable
43  fun ControlButton() {
44      var isOn = false
45
46      Button(onClick = {
47          isOn = !isOn
48          Log.d(TAG,  msg: "Button on: $isOn ")
49      }) {  this: RowScope
50          if(isOn) {
51              Text(text = "Off")
52          }
53          else {
54              Text( text: "On")
55          }
56      }
57  }
```



State

```
42  @Composable
43  fun ControlButton() {
44      //var isOn = false
45      var isOn by remember {
46          mutableStateOf( value: false)
47      }
48
49      Button(onClick = {
50          isOn = !isOn
51          Log.d(TAG,  msg: "Button on: $isOn ")
52      }) {  this: RowScope
53          if(isOn) {
54              Text(text = "Off")
55          }
56          else {
57              Text( text: "On")
58          }
59      }
60  }
```



State

rememberSaveable

- Husker på tværs af konfigurationer (så som skærm rotation)

Restoring state in Compose

Use `rememberSaveable` to restore your UI state after an activity or process is recreated. `rememberSaveable` retains state across recompositions. In addition, `rememberSaveable` also retains state across activity and process recreation.

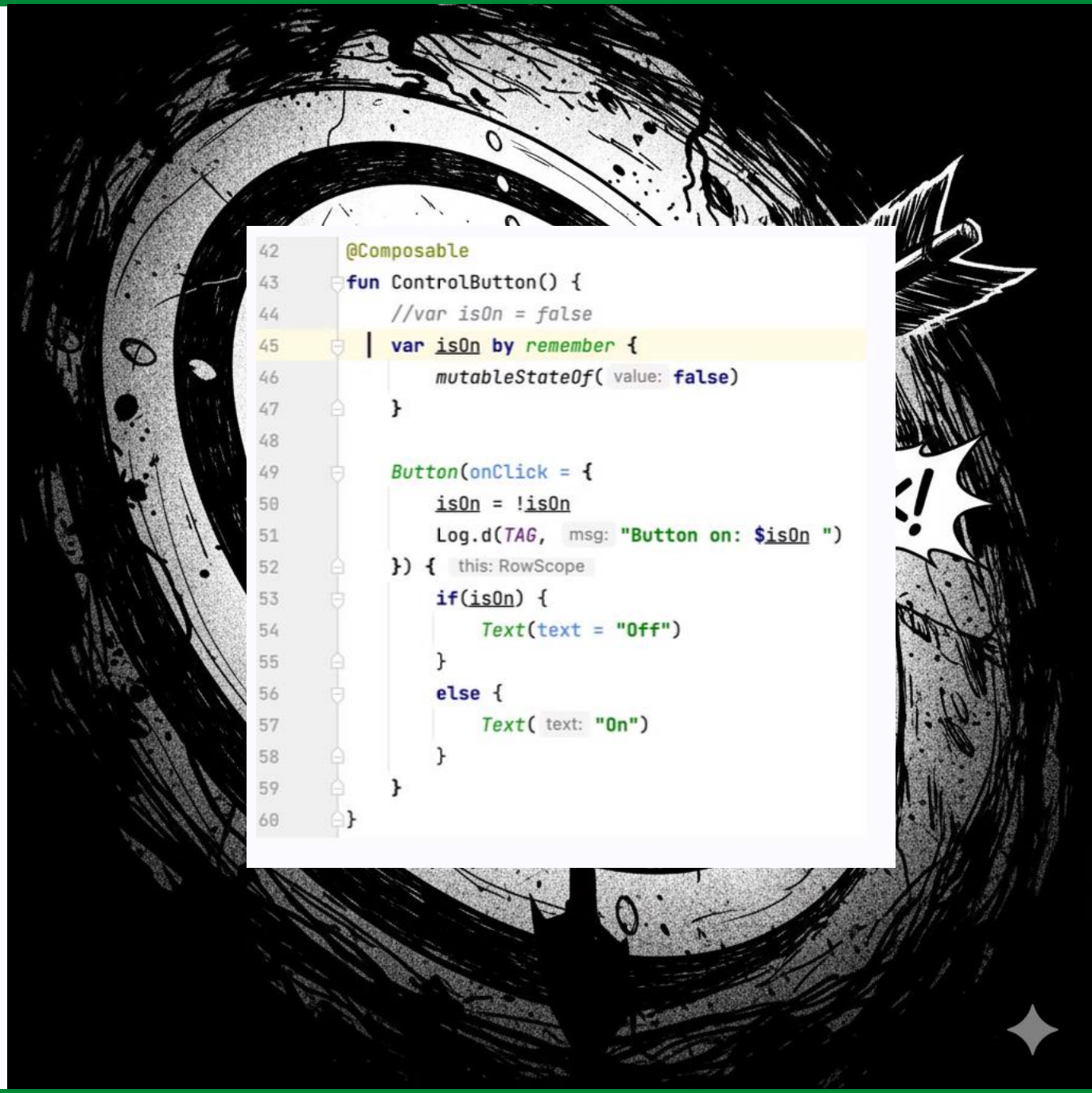


Øvelse 1

Du vil gerne løfte state ud af en composable for at gøre den “stateless”, også kaldt **State Hoisting**.

- Hvordan kunne man gøre ControlButton stateless?
- Er det en god ide at gøre composables stateless?

Sammen 2 og 2 / 5 minutter



```
@Composable
fun ControlButton() {
    //var isOn = false
    var isOn by remember {
        mutableStateOf( value: false)
    }

    Button(onClick = {
        isOn = !isOn
        Log.d(TAG, msg: "Button on: $isOn ")
    }) { this: RowScope
        if(isOn) {
            Text(text = "Off")
        }
        else {
            Text( text: "On")
        }
    }
}
```

Mobile Software Arkitektur

Øvelse 2

- Separation of concerns

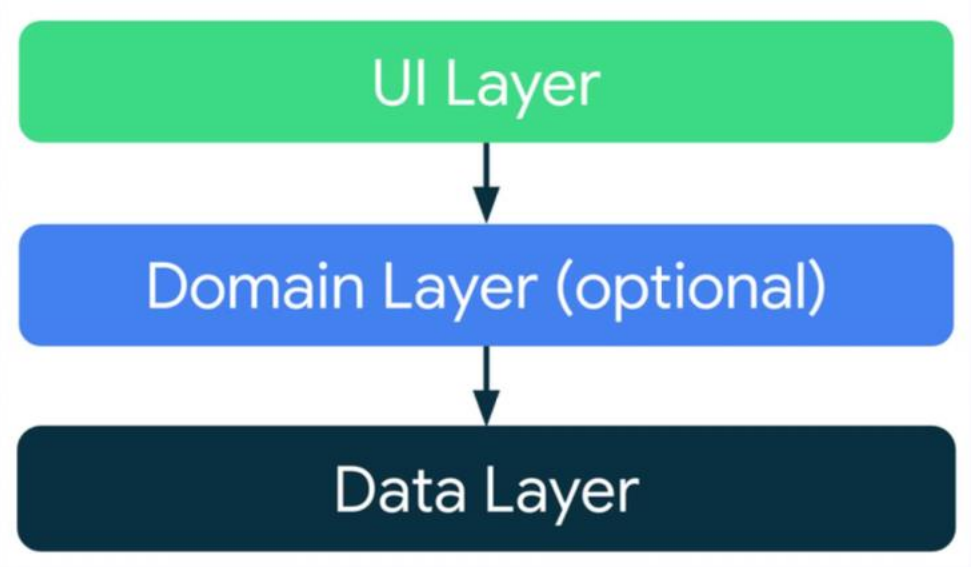
- Drive UI og opdatering af UI fra en model

Diskuter jeres forståelse af de to ovenstående koncepter.

Sammen 2 og 2 / 5 minutter



UI laget



UI laget

- Viser applikations data
- Primære måde at interagere med dataen på

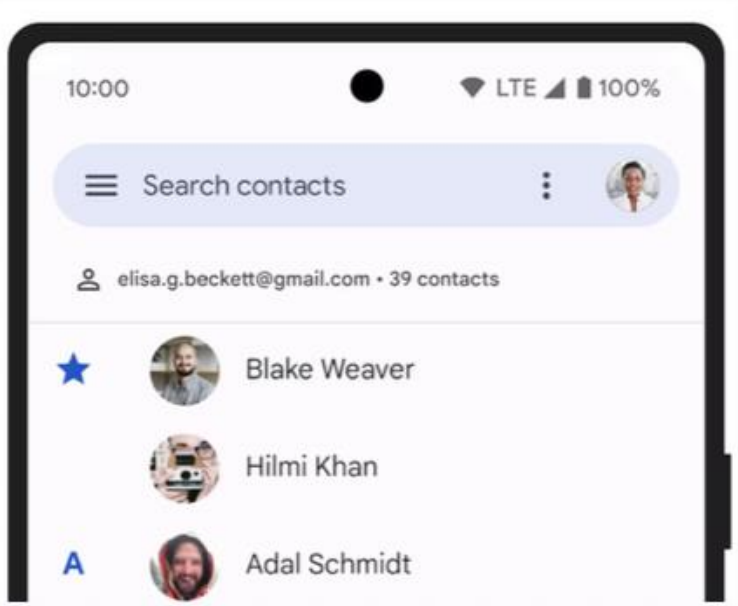
Når dataen bliver ændret enten via interaktion(så som at trykke på en knap) eller eksternt input (som foreksempel hvis data sendes over nettet) skal brugergrænsefladen opdateres så den reflekterer ændringen I data

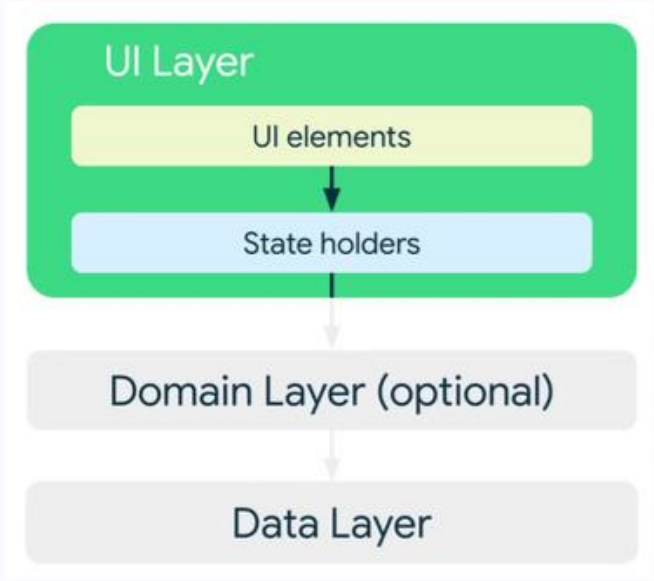
UI laget



UI laget

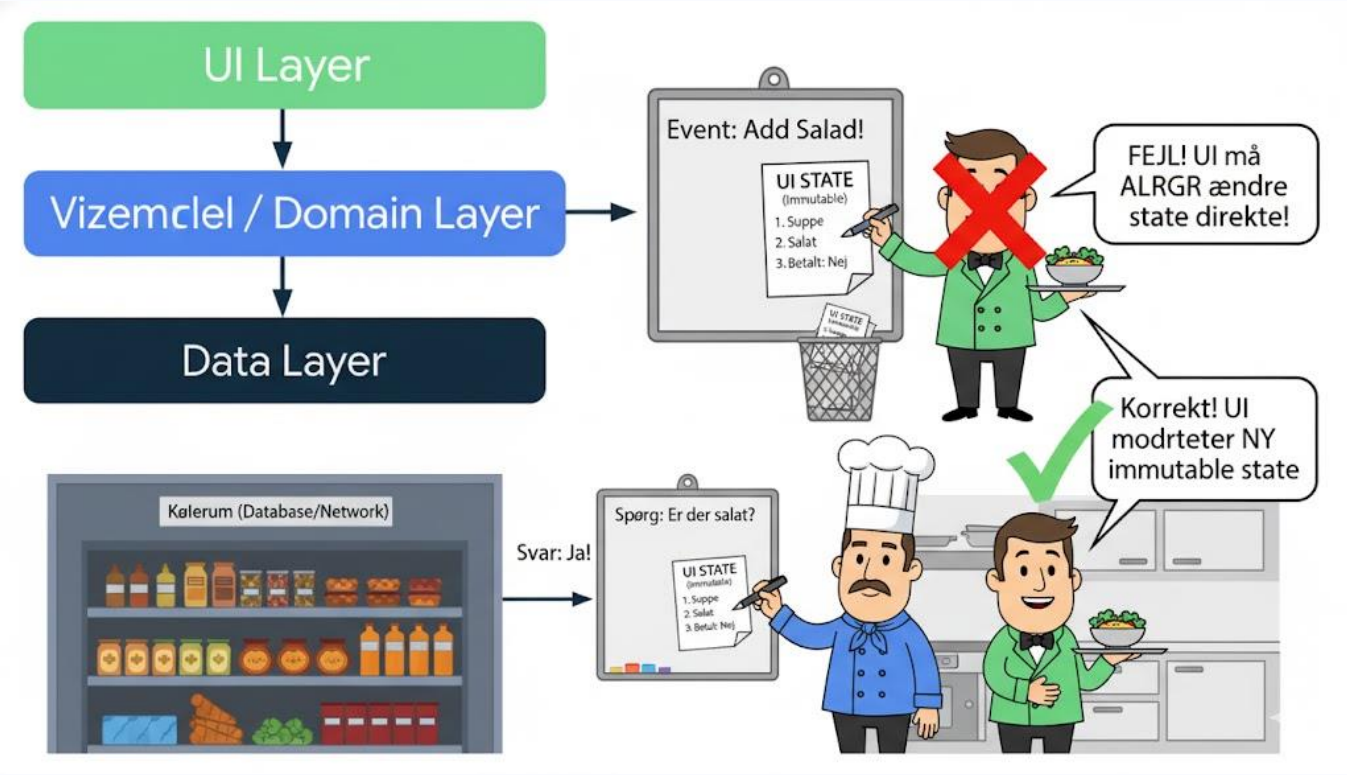
- State og data kommer fra data laget
- Applikationens data opbevares oftest i et andet format end det UI laget konsumerer





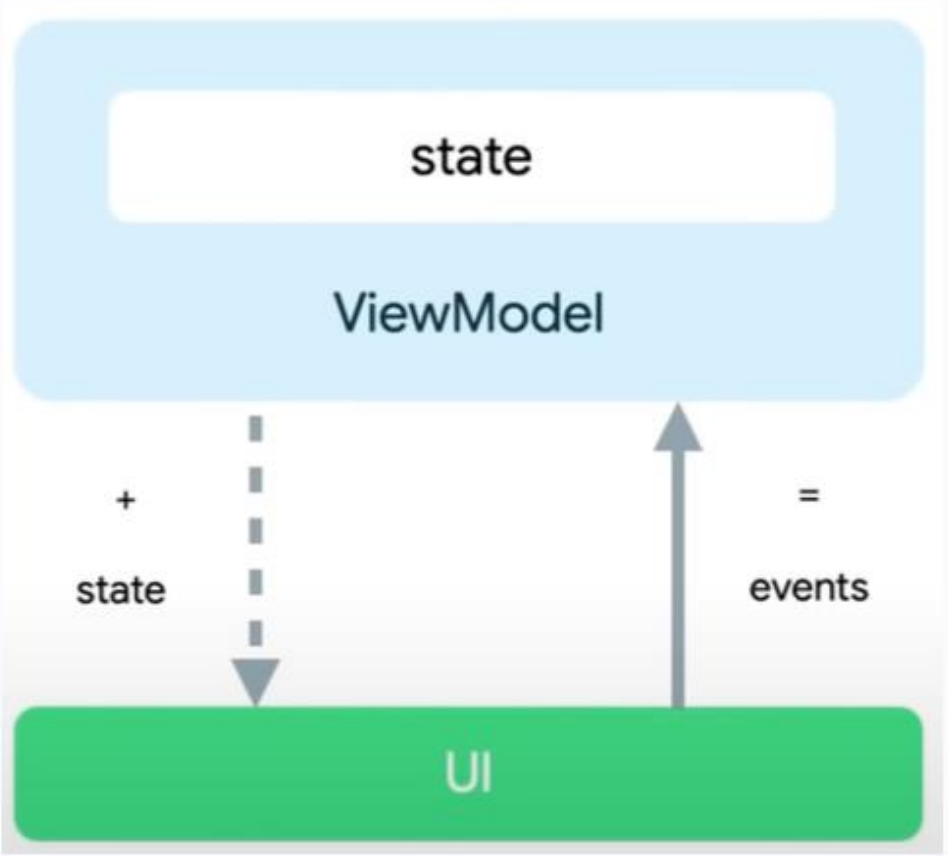
UI State er immutable

- Din UI må **ALDRIG** selv ændre i de data, den får. Den skal kun **kigge** på dataene og **rapportere**, når brugeren vil lave en ændring.
- Brugergrænsefladen repræsenterer et immutable øjebliksbillede af den underliggende data



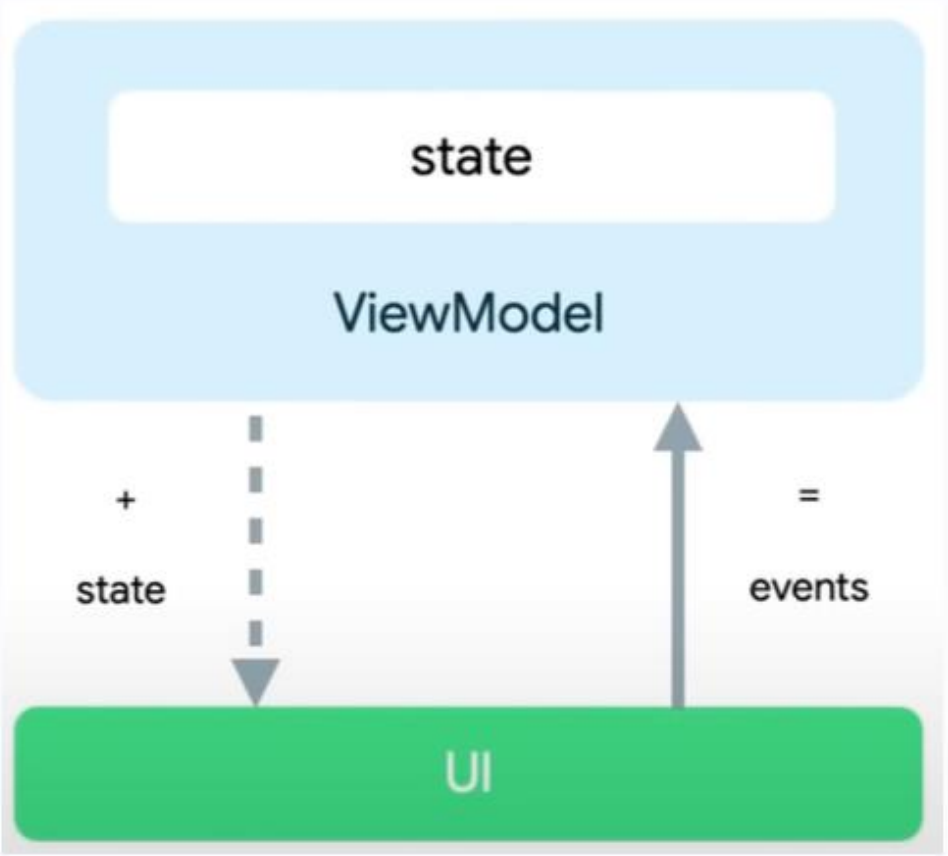
Ensrettet Dataflow (Unidirectional Data Flow)

- **tilstand (state)** flyder kun i én retning, typisk fra en forældrekomponent til en børnekomponent. De **events**, der ændrer dataene, flyder i den modsatte retning.
- Opretholder bedre datakonsistens, er mindre udsat for fejl, er lettere at fejlfinde og giver alle fordelene ved **SSOT**-mønstret.
- I Android flyder tilstand eller data normalt fra de "højere" niveauer af hierarkiet til de "lavere" (f.eks. fra Data-laget til UI-laget).



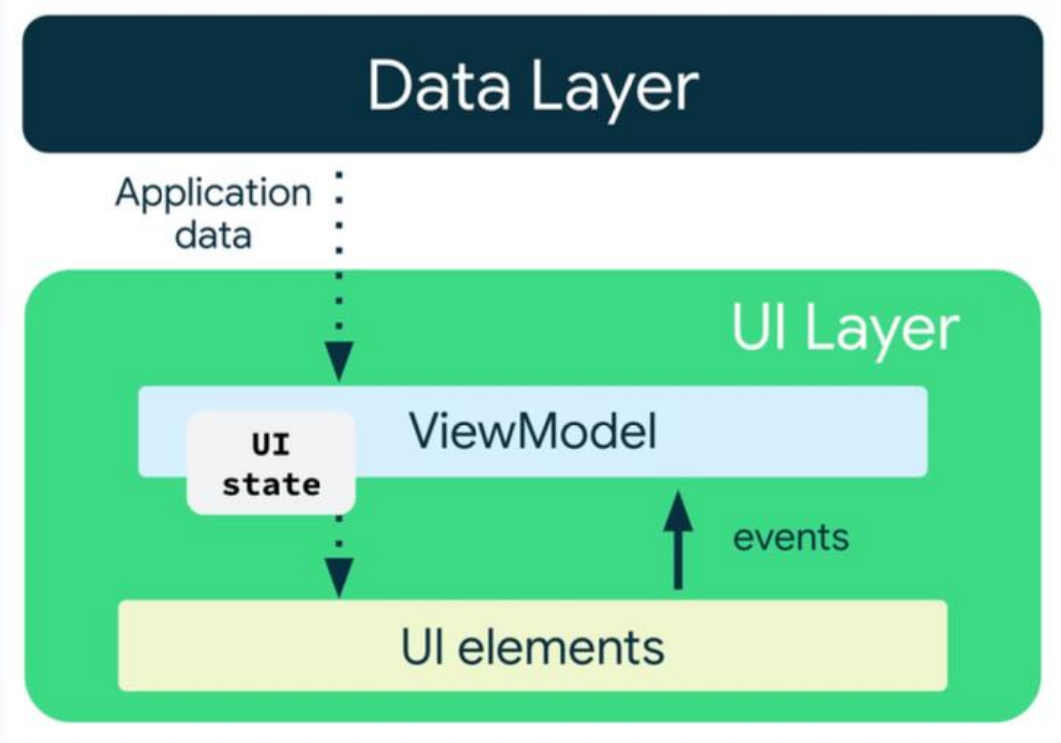
En Kilde til Sandhed (Single Source of Truth)

- Når du definerer en ny type data i din app, skal du tildele den en **Enkel Kilde til Sandhed (SSOT)**. SSOT'en er **ejeren** af disse data, og **kun SSOT'en må ændre** dem.
- For at opnå dette, præsenterer SSOT'en dataene ved hjælp af en **uforanderlig (immutable) type**. For at ændre dataene, tilbyder SSOT'en funktioner eller modtager "events" (begivenheder), som andre dele af appen kan kalde.
- Gør data **centraliseret** og **sporbar**



Ensrettet Dataflow

- UI ændrer aldrig på data/state men sender events ned til data laget som sender ny state (recomposition) tilbage.
- Data/state i UI er oftest transformeret

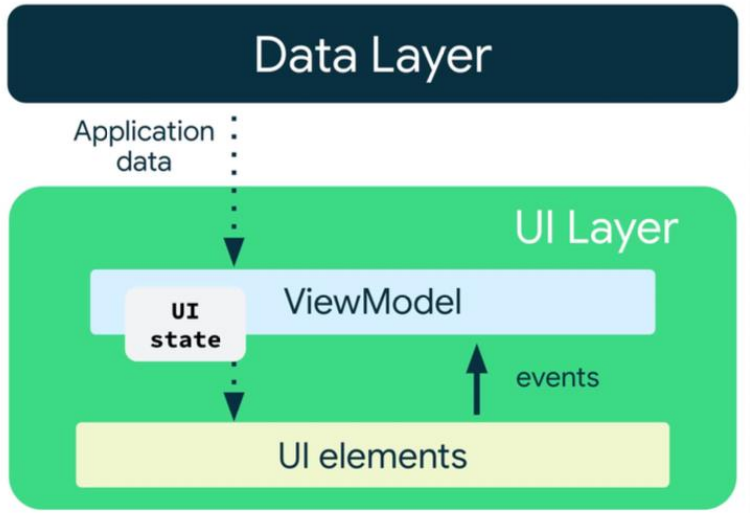


ViewModel

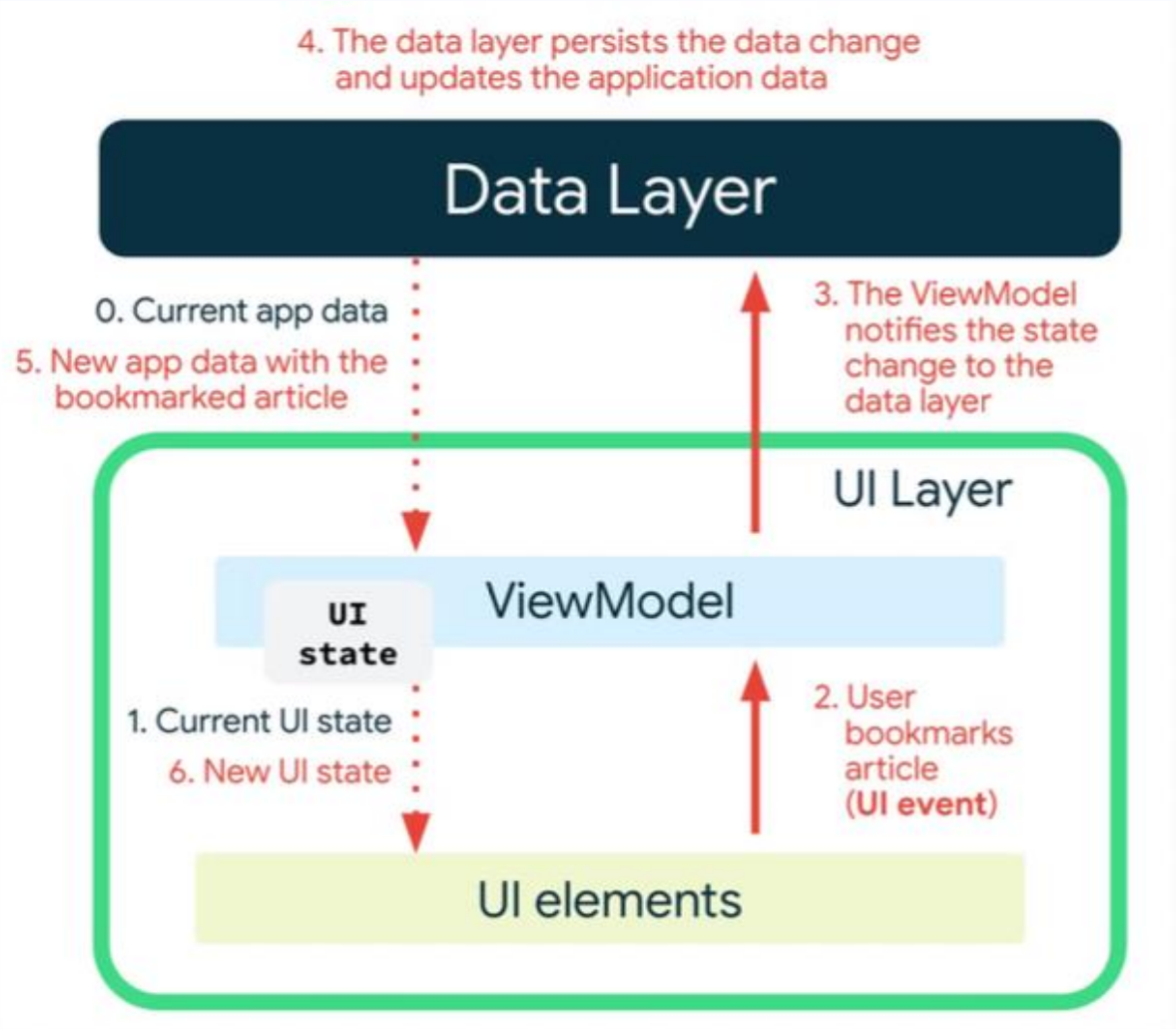
- Holder og eksponerer den state der skal konsumeres af brugergrænsefladen
- Brugergrænsefladens state er applikations data der er transformeret af ViewModel
- Brugergrænsefladen opdaterer ViewModel om brugers interaktion
- ViewModel håndterer brugeres handlinger og opdaterer applikationens state

Ændringerne propageres tilbage til brugergrænsefladen via:

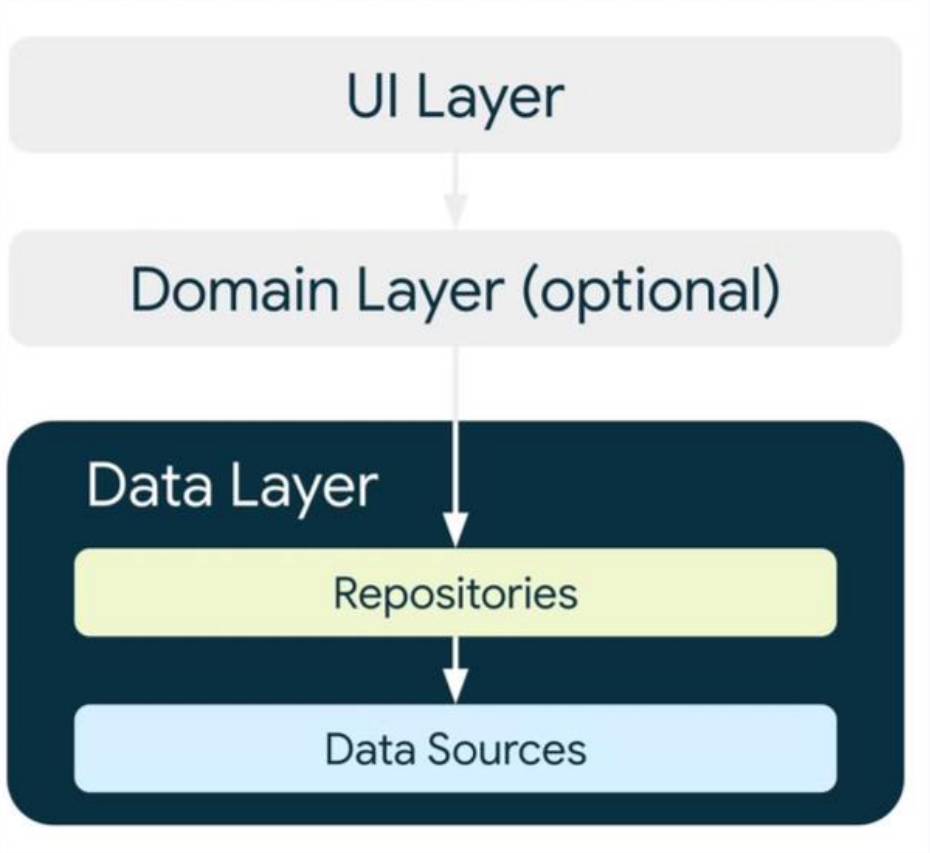
- observer pattern og recomposition



Ensrettet Dataflow



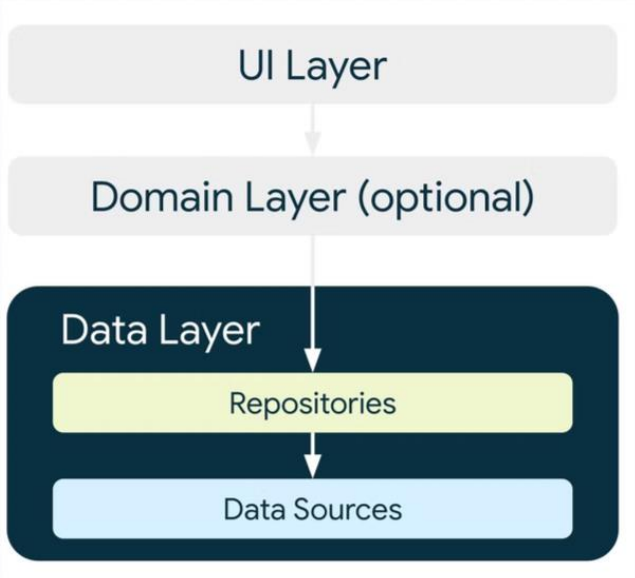
Data Lag


UI Layer
Domain Layer (optional)
Data Layer
Repositories
Data Sources

Repositorier(Repository) klasser

Ansvarlig for:

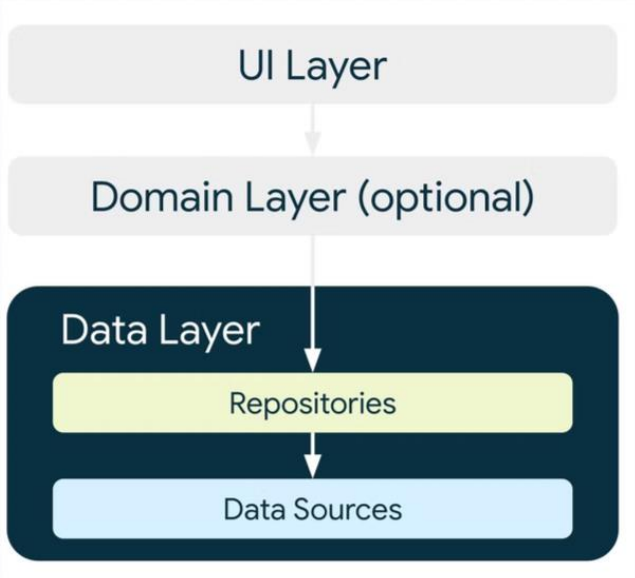
- Gøre data tilgængelig
- Centraliserer ændringer i data (SSOT)
- Løse konflikter mellem flere datakilder (lokal vs database/netværk)
- Abstraherer datakilderne fra resten af appen



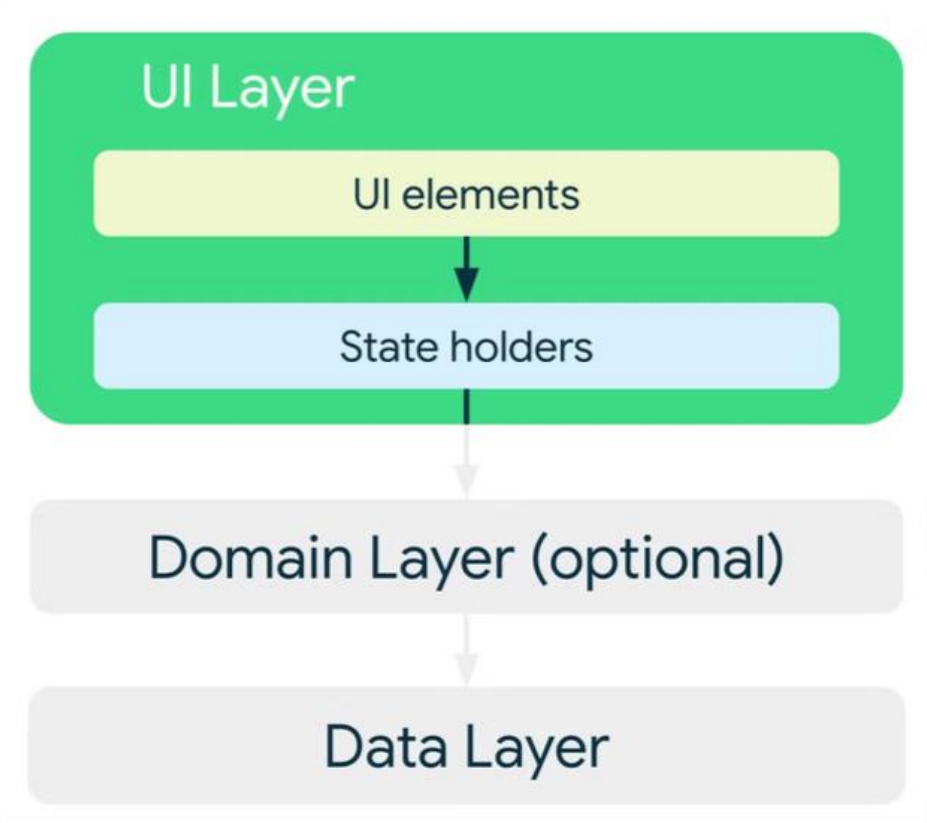
Datakilder(Data sources)

Specifikke klasser, der håndterer **én** enkelt kilde til data:

- De "rigtige" steder, data fysisk hentes fra eller gemmes til.
- En Data Source er typisk "dum" – den henter eller gemmer bare, hvad den får besked på.
- et Repository vil typisk koordinere og tale med én eller flere Data Sources.



Model – View – View Model (MVVM)



Flows

Hvad er et Flow?

- Et **Flow** (en "strøm") er en asynkron datastrøm fra **Kotlin Coroutines**.
- Den kan levere **flere værdier** over tid, én efter én.
- Erstatter Callbacks
- **Kotlin Coroutines** er et letvægtsmekanisme til asynkron programmering i Kotlin. (mere senere)



Hvad bruger vi et Flow til

- **Asynkron Håndtering** af netværkskald eller databaseadgang uden at blokere hovedtråden (UI-tråden).
- **Ensrettet Dataflow (UDF):** Flow er selve "røret", som data flyder igennem:
  - Dit **Repository** *udsender* (emits) et Flow<List<Product>>.
  - Din **ViewModel** *opsamler* (collects) dette Flow.
  - Din **UI (Compose)** *observerer* (collects) et StateFlow fra din ViewModel.
- **Struktureret Samtidighed (Structured Concurrency):** Et Flow aflyses automatisk, når den Coroutine, der startede det, aflyses (f.eks. når brugeren forlader skærmen



Kolde og varme Flow

Cold Flow (Kold Strøm) - *Eksempel: flow { ... }*

•**Analogien:** En vandhane. Den er slukket, og der sker intet, før du åbner for den.

•**Definition:** Et "koldt" flow er **passivt**. Det starter først sin "producer"-kode (f.eks. netværkskaldet), når nogen kalder .collect() på det.

•Hver ny lytter (collector) starter et nyt, uafhængigt flow

Hot Flow (Varm Strøm) - *Eksempel: StateFlow, SharedFlow*

•**Analogien:** En radiostation. Den sender sit signal ud hele tiden, uanset om nogen har tændt for radioen.

•**Definition:** Et "varmt" flow er **aktivt**. Det producerer værdier, selvom ingen lytter.

•Når en ny lytter tilmelder sig, får de den seneste værdi (eller ingenting) og alle fremtidige værdier.

StateFlow

- StateFlow er en speciel type **varmt flow**, designet til at holde **UI-tilstand (UI State)**.
- Den er den primære "State Holder" i din ViewModel.
- **Kerneegenskaber:**
  - Den har *altid* en værdi (en startværdi).
  - Den udsender kun nye værdier, hvis de er forskellige fra den forrige.
  - Når en ny lytter (f.eks. din UI) tilmelder sig, får den **øjeblikkeligt** den seneste værdi.

```
// Den private, muérbare version
private val _uiState = MutableStateFlow(UiState(isLoading = true))

// Den offentlige, uforanderlige version til UI'en
val uiState: StateFlow<UiState> = _uiState.asStateFlow()
```

StateFlow

StateFlow

`StateFlow` is a data holder observable flow that emits the current and new state updates. Its `value` property reflects the current state value. To update state and send it to the flow, assign a new value to the value property of the `MutableStateFlow` class.

In Android, `StateFlow` works well with classes that need to maintain an observable immutable state.

UI ”lytter” til StateFlow

- Din Composable UI skal bruge et State-objekt for at vide, hvornår den skal recompose.
- Du omdanner et StateFlow (fra din ViewModel) til et Compose State med en simpel funktion.
- **Mest almindelige metode:** collectAsStateWithLifecycle()
  - Denne funktion er "lifecycle-aware" (livscyklus-bevidst).
  - Den opsamler kun værdier, når din app er i forgrunden, og **stopper automatisk** opsamlingen, når appen går i baggrunden (hvilket sparer batteri og ressourcer).

```
@Composable
fun MyScreen(viewModel: MyViewModel = viewModel()) {
    // uiState er nu et Compose State-objekt
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()

    // Nu kan du bruge værdierne direkte
    Text(text = "Hello ${uiState.userName}")
}
```

Flow: Operatorer (Data-transformation)

Flows sande styrke er dets **operatorer**.

- Det er funktioner, der lader dig transformere datastrømmen på vejen.
- Dette sker typisk i din ViewModel eller Repository.

Almindelige operatorer:

- **map**: Transformér hver værdi. (F.eks. ProductData -> ProductUiModel).
- **filter**: Fjern værdier, der ikke opfylder et krav. (F.eks. filter { it.isInStock }).
- **combine**: Kombinér to eller flere flows til ét. (F.eks. kombinér et searchQueryFlow med et allProductsFlow for at skabe et filteredProductsFlow).
- **catch**: Håndtér fejl (exceptions) i strømmen elegant.

Flow

There are three entities involved in streams of data:

- A **producer** produces data that is added to the stream. Thanks to coroutines, flows can also produce data asynchronously.
- **(Optional) Intermediaries** can modify each value emitted into the stream or the stream itself.
- A **consumer** consumes the values from the stream.

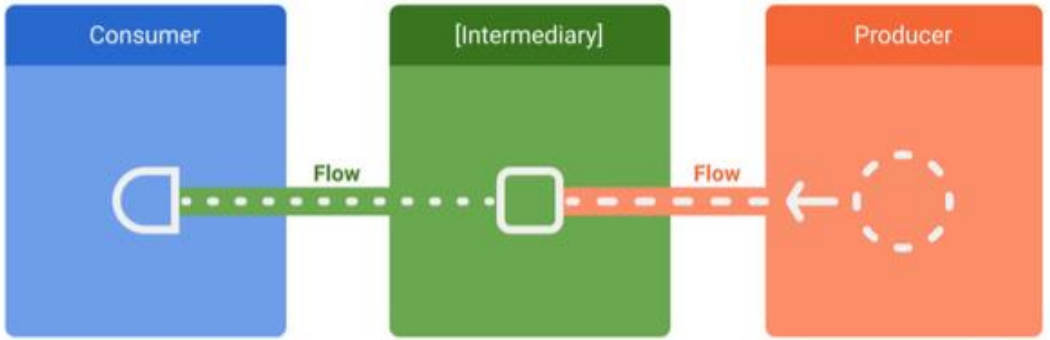


Figure 1. Entities involved in streams of data: consumer, optional intermediaries, and producer.

In Android, a *repository* is typically a producer of UI data that has the user interface (UI) as the consumer that ultimately displays the data. Other times, the UI layer is a producer of user input events and other layers of the hierarchy consume them. Layers in between the producer and consumer usually act as intermediaries that modify the stream of data to adjust it to the requirements of the following layer.

[Kotlin flows on Android | Android Developers]

Parallel programmering

Øvelse 3

Parallel computing (parallel databehandling) er et programmerings paradigme, hvor mange beregninger eller processer udføres samtidigt.

Vi skelner mellem 2 typer af parallel databehandling:

- **Parallellisme**
- **Samtidighed (Concurrency)**

Sammen 2 og 2 / 5 minutter



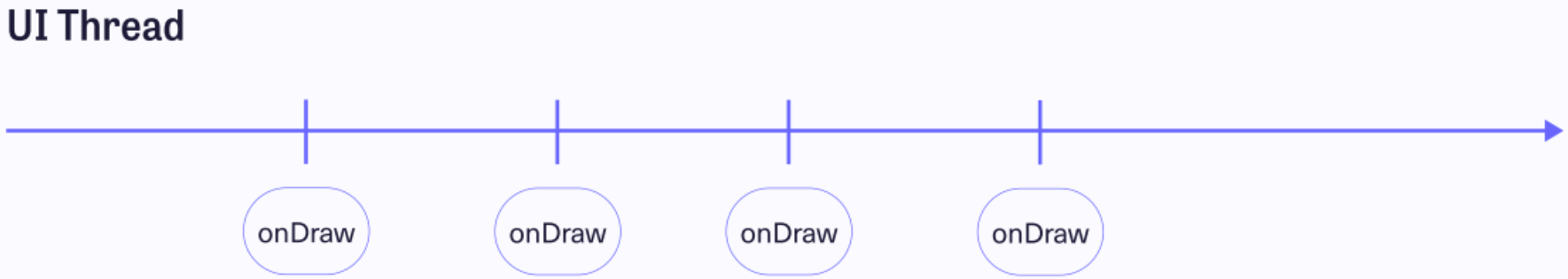
Øvelse 4

Er der nogle ulemper ved concurrency ?

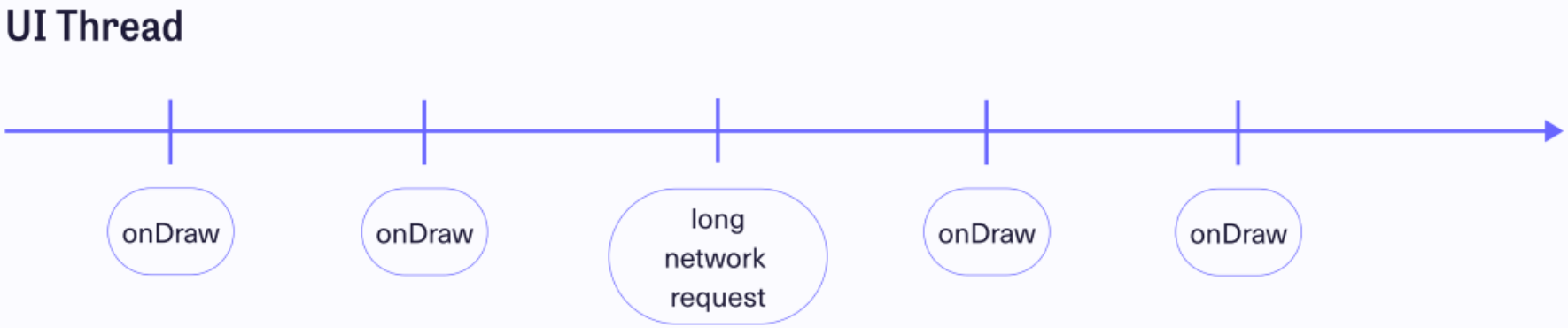
Sammen 2 og 2 / 5 minutter



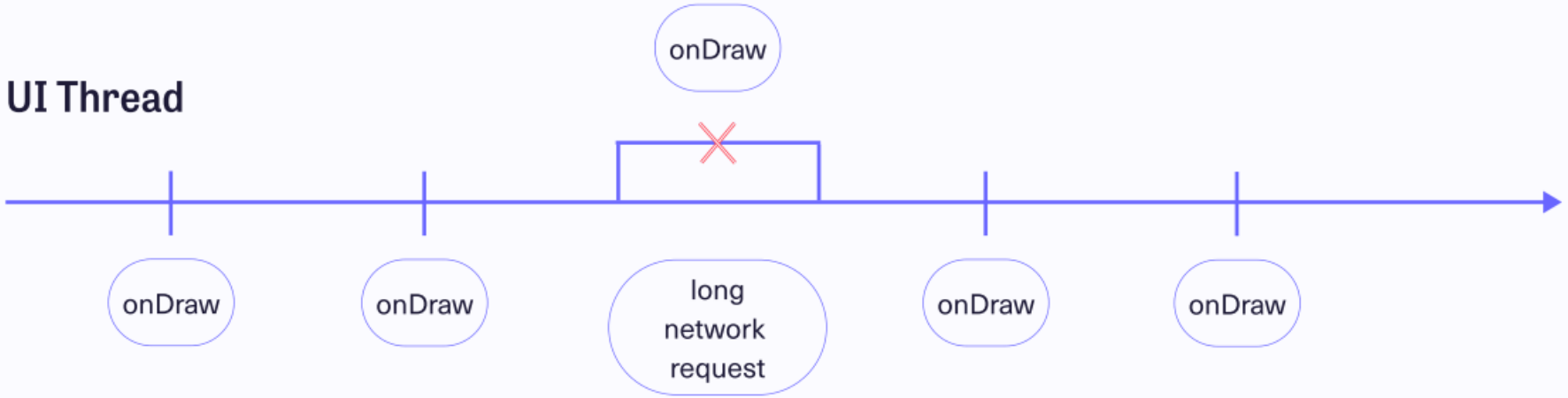
concurrency



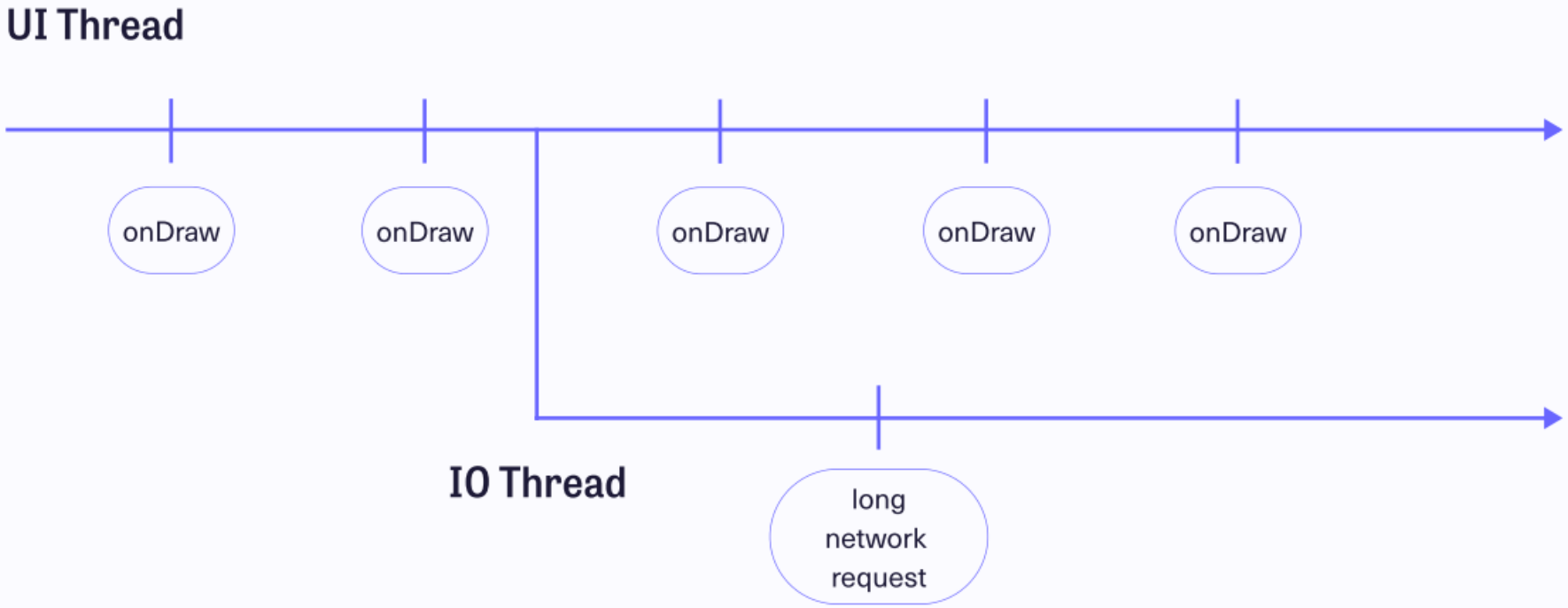
concurrency



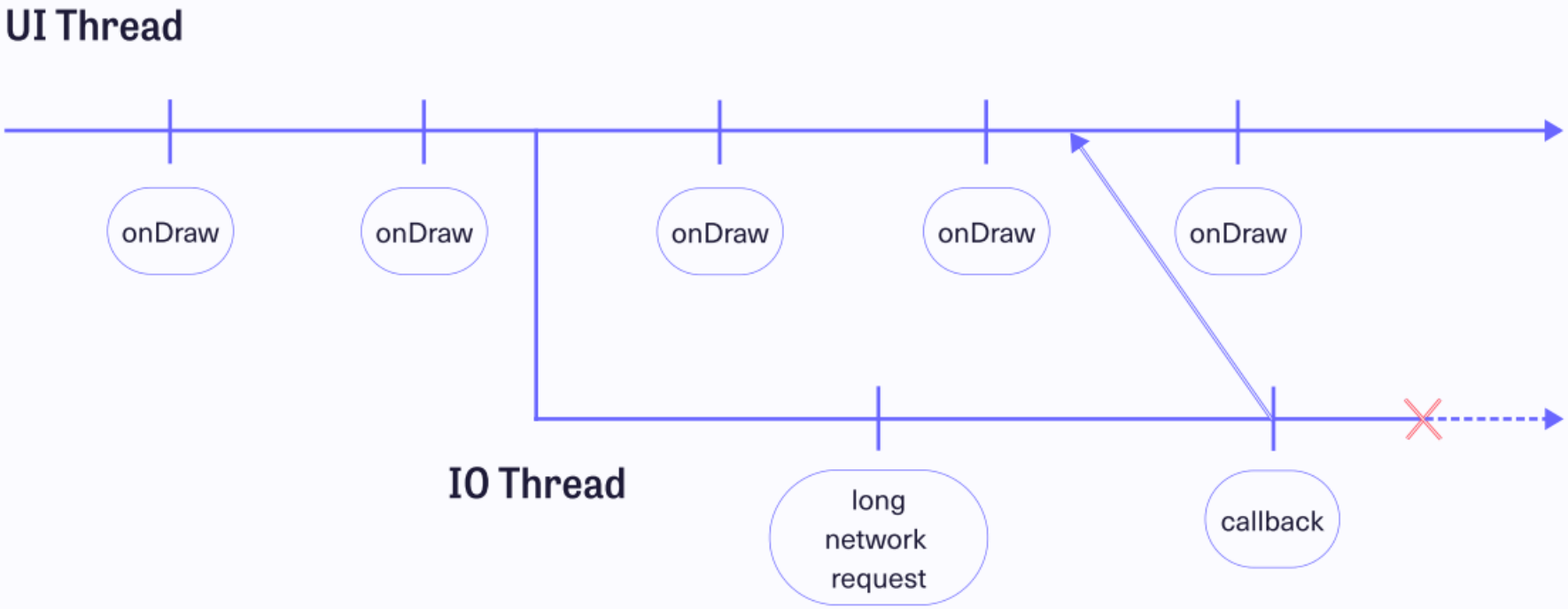
concurrency



concurrency

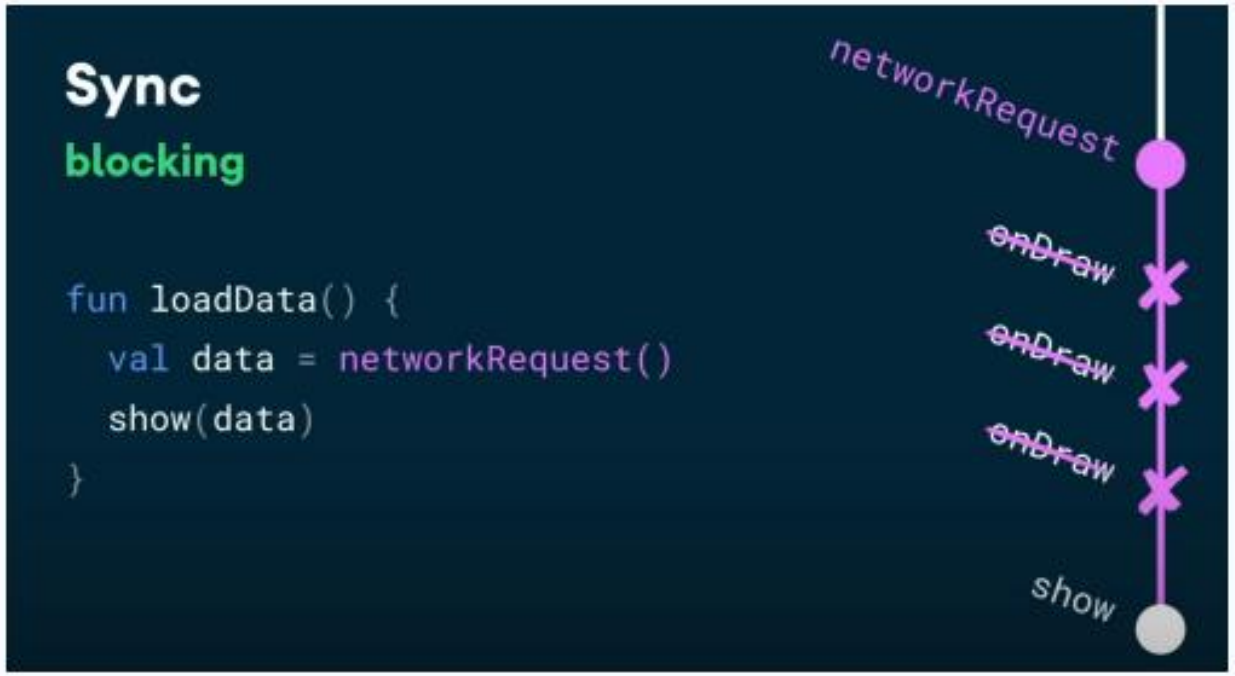


concurrency



Kotlin Coroutines

Kotlin coroutines





Synkron (blokerende) kode



Synkron (blokerende) kode

Potentiel løsning:

- Flyt koden over på en anden tråd
- Brug et callback



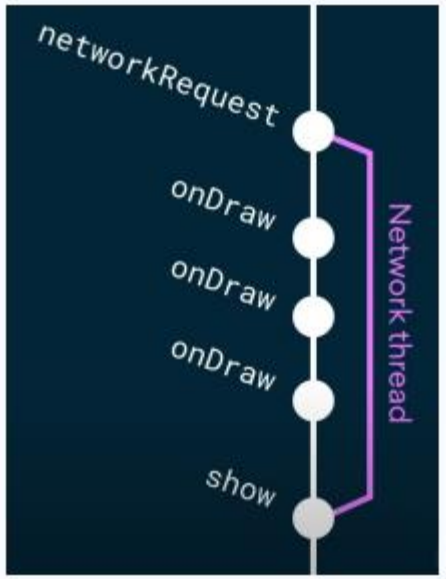
Asynkron kode med callback

Async
with callbacks

```
fun loadData() {
  networkRequest { data ->
    show(data)
  }
}
```

Async
with callbacks

```
fun networkRequest(onSuccess: (Data) -> Unit) {
  DefaultScheduler.execute {
    // Blocking network request code
    postToMainThread(onSuccess(result))
  }
}
```



Asynkron kode med callback

Async

with callbacks

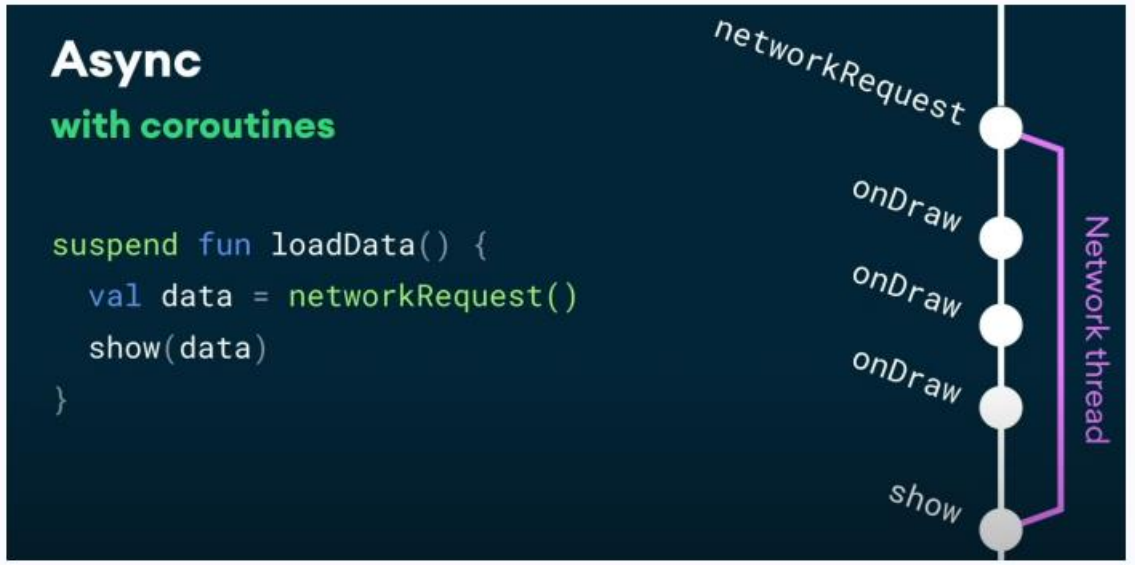
```
fun loadData() {
  networkRequest { data ->
    anotherRequest(data) { otherData ->
      // ...
    }
  }
}
```



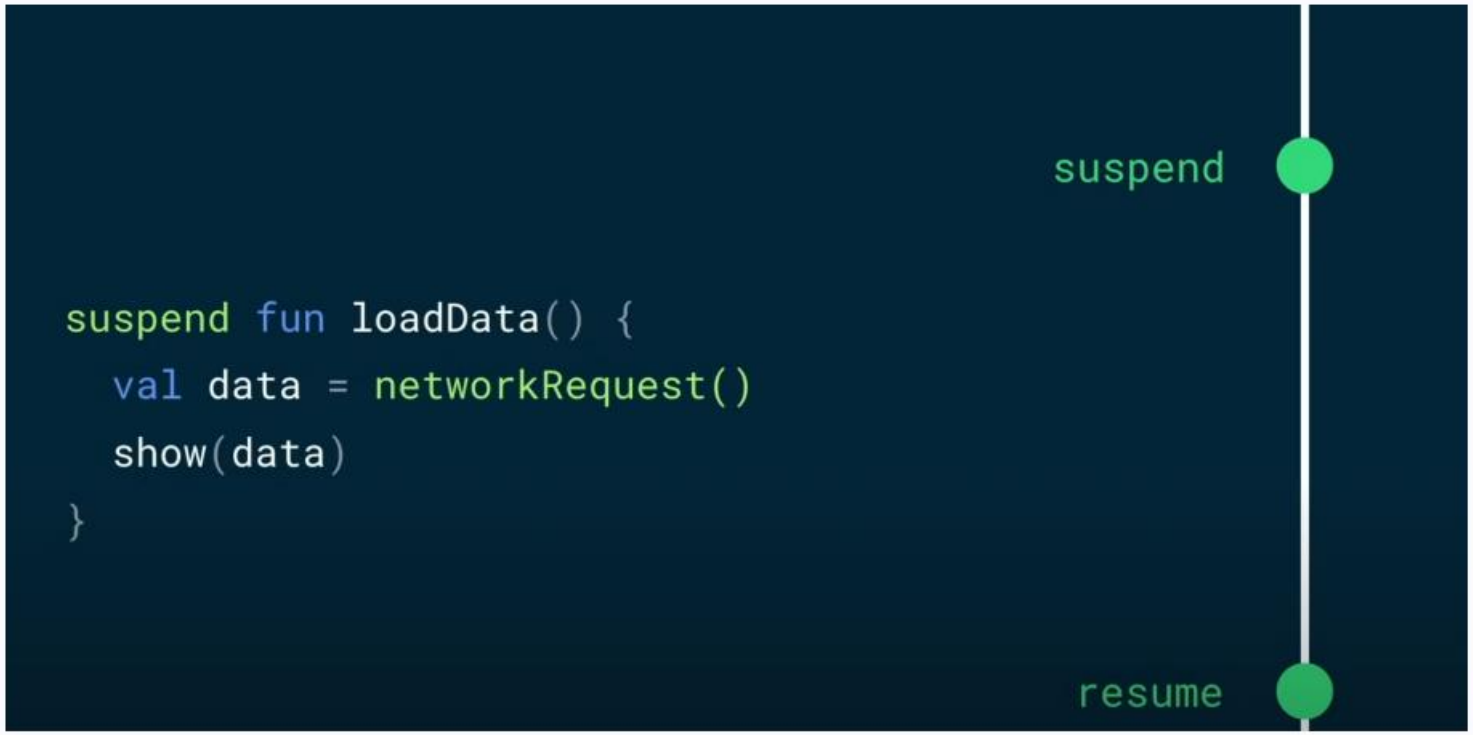
Coroutines

En anden potentiel løsning:

- Simplificer asynkron programmering
- Koden kan læses sekventielt



Suspended funktioner



Suspended funktioner

```
suspend fun loadData() {
  val data = networkRequest()
  show(data)
}

suspend fun networkRequest(): Data =
  withContext(Dispatchers.IO) {
      // Blocking network request code
  }
```

Hvad er Kotlin Coroutines?

Coroutine basics:

“A coroutine is an instance of suspendable computation. It is conceptually similar to a thread, in the sense that it takes a block of code to run that works concurrently with the rest of the code.”

"Coroutines can be thought of as light-weight threads, but there is a number of important differences that make their real-life usage very different from threads."

Hvad er Kotlin Coroutines?

1. "En asynkron programmeringsmodel"

•**Hvad betyder det?** Det er en **struktur** eller **metode** til at håndtere opgaver, der tager tid (som netværkskald eller databaseadgang).

•**Hvorfor?** Målet er at køre disse opgaver **væk fra hovedtråden (UI-tråden)**.

•**Resultat:** Dette forhindrer, at din app "fryser" eller går ned, og sikrer en flydende brugeroplevelse.

2. "En abstraktion oven på tråde (threads)"

•**Hvad betyder det?** Du behøver ikke at styre Threads manuelt. Coroutines er en **meget "billigere" og mere letvægts** enhed end en tråd.

•**Hvordan?** Én enkelt tråd kan køre tusindvis af coroutines. En coroutine kan **"suspendere"** (holde pause), mens den venter (f.eks. på et netværkssvar), og lade andre coroutines køre på den samme tråd i mellemtiden.

•**Resultat:** Det er meget mere effektivt og fjerner kompleksiteten ved manuel trådhåndtering.

Coroutines

- **suspend:**
  - suspender eksekveringen af den nuværende coroutine og gem alle lokale variable

- **resume:**
  - Fortsæt den suspenderede coroutine fra dér hvor den blev sat på pause

- Kan kun kalde suspenderede funktioner ved at bruge en coroutine builder til at ”starte en ny coroutine”

Hvad bruges coroutines til

- Opgaver der tager lang tid at løse og kan blokere ui/main thread
- Main Sikkerhed
- Læsbarhed af kodebasen (fordi koden kan læses sekventielt)

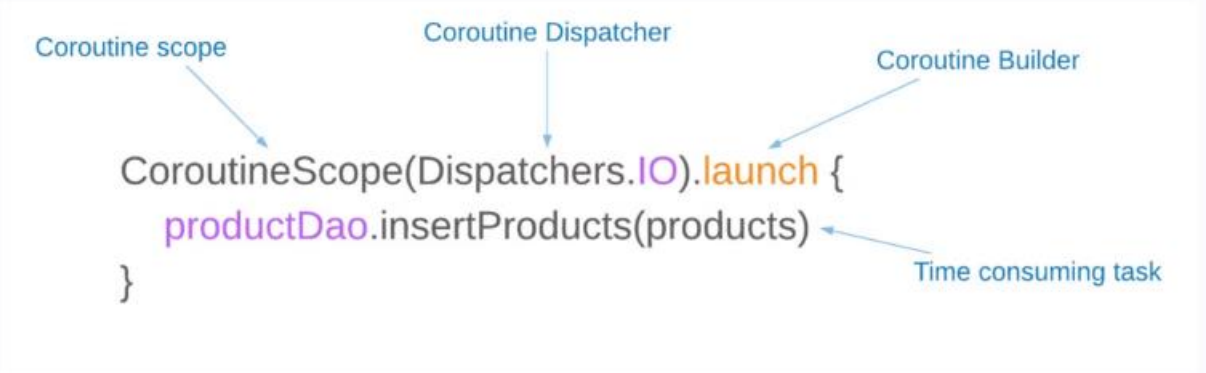
Hvordan bruges coroutines

- **Definér et CoroutineScope (Hvor/Hvornår):** Først defineres et CoroutineScope. Dette er "containeren", der styrer coroutinens livscyklus (f.eks. hvornår den skal annulleres).
- **Vælg en CoroutineDispatcher (Hvilken tråd):** Scopet tildeles en Dispatcher. I eksemplet er det Dispatchers.IO, som er en speciel pulje af tråde optimeret til I/O-operationer (Input/Output). Dette flytter opgaven væk fra UI-tråden.



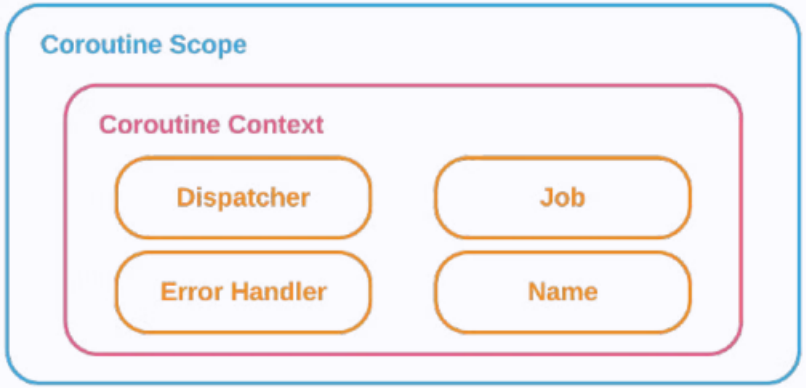
Hvordan bruges coroutines

- **Brug en CoroutineBuilder (Start):** Derefter bruges en "builder" som .launch { ... } til at starte selve coroutinen. launch bruges til opgaver, hvor du ikke har brug for at få et resultat tilbage (også kaldet "fire-and-forget").
- **Udfør den tidskrævende opgave (Hvad):** Inde i launch-blokken placerer du den kode, der tager tid.



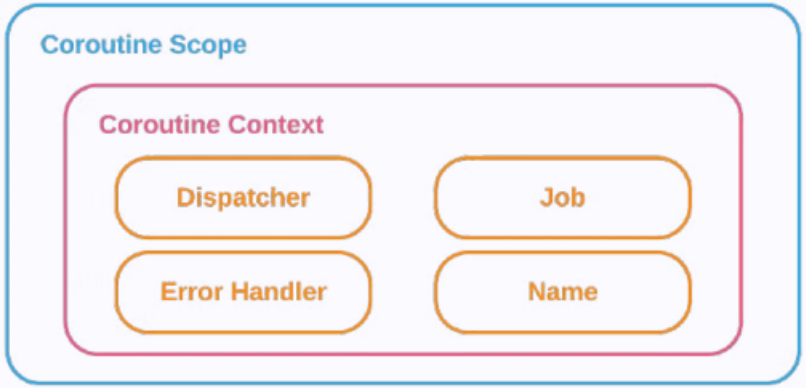
Coroutine Scope

- Coroutines kører i et **coroutine scope** - ved hjælp af launch
- Holder styr på coroutines
- Bestemmer coroutinens levetid



Coroutine Scope

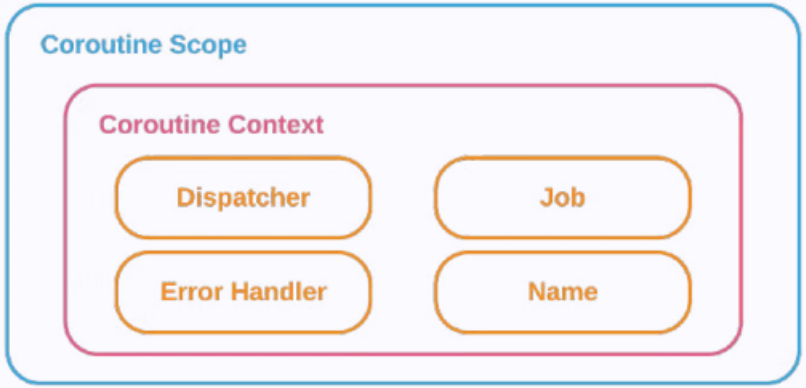
- Definerer en coroutines livscyklus/levetid
- **CoroutineScope**
  - Opretter et brugerdefineret scope. Skal annulleres manuelt.
- **viewModelScope**
  - Er bundet til levetiden af en ViewModel.
- **lifecycleScope**
  - Er bundet til levetiden af en LifecycleOwner (f.eks. en Activity).



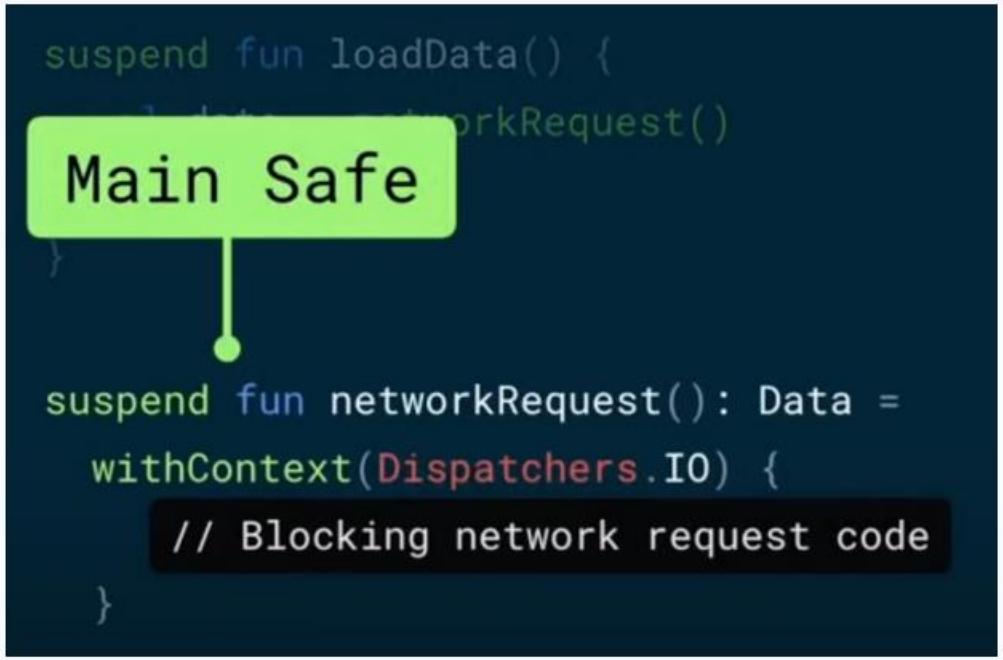
Coroutine Context

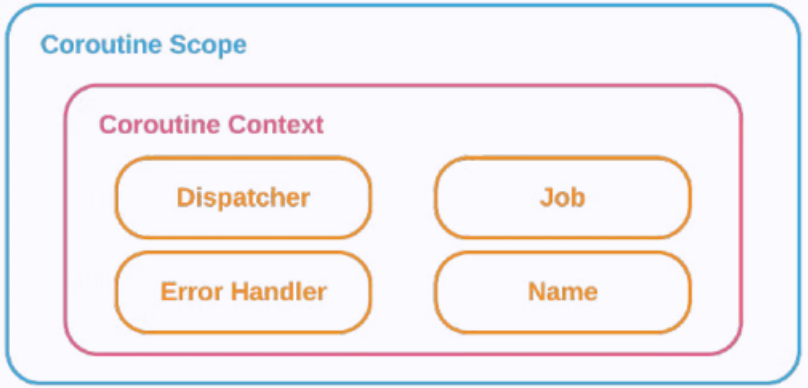
En samling af "kontekst"-elementer:

- **Dispatcher** (hvilken tråd coroutinen skal køre på)
- **Job** (kontrollerer coroutinens livscyklus)
- **Error handler** (håndterer fejl, når en coroutine kaster en exception)
- **Name** (navn på coroutinen)



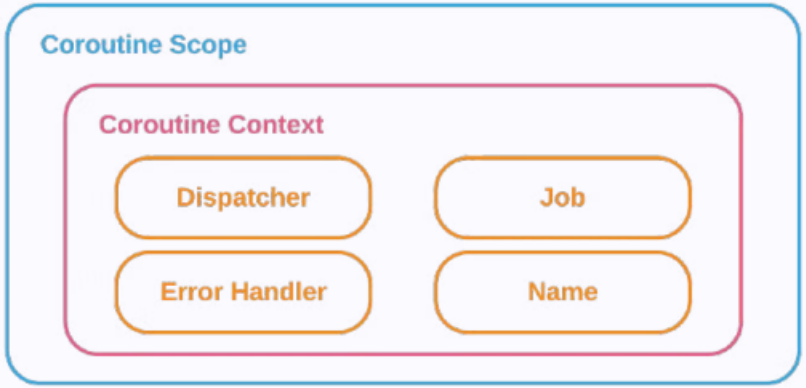
Dispatcher


suspend fun loadData() {
Main Safe
}
suspend fun networkRequest(): Data =
withContext(Dispatchers.IO) {
// Blocking network request code
}


Coroutine Scope
Coroutine Context
Dispatcher
Job
Error Handler
Name

Dispatcher

- **Dispatcher.Main**
  - UI/ikke blokerende
- **Dispatchers.IO**
  - Til netværk eller disk
- **Dispatchers.Default**
  - Til CPU intensive opgaver



Opgave til resten af dagen

Byg din første KMP applikation

- Følg opgaveseddel på Learn og byg videre på den applikation i startede på sidste uge.